

Path Planning

Jett Vanbrocklin

March 2026

1 Introduction

The path of a body in travel from a starting point to an ending point can be modeled by the connection of discrete points in space. A method of engineering the connection of discrete points is to map the environment, its boundaries, and the obstacles within those boundaries into discrete regions. Information that can be used to determine whether a given region of space within the environment can be inhabited by the body.

There are several algorithmic solutions to generate a path given the information described above. This report follows the analysis and implementation of three: Depth First Search (DFS), Dijkstra's Algorithm, and A-star (A*).

Additionally, the continuous paths determined by the algorithms only provide available positional components as output. However, the simulator (described thoroughly in [2], [3], and [4]) requires a time history of desired position, velocity, acceleration, and yaw. To sort this issue, and optimize the path for travel time, polynomial smoothing must be used. A section in this report analyzes the effect of polynomial smoothing under various parameters.

2 Analysis

2.1 Depth First Search Path Analysis

While Depth First Search can find a path from a starting node to an ending node, it is not guaranteed that it will be the shortest path. The following example showcases this, where the returned path is not the shortest path.

Consider a 5-by-5 occupancy grid of the following:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Where 0's represent open nodes and 1's represent obstacles.

The path found by a DFS algorithm and the corresponding metrics are displayed respectively in **Figure 1** and **Table 1** below:

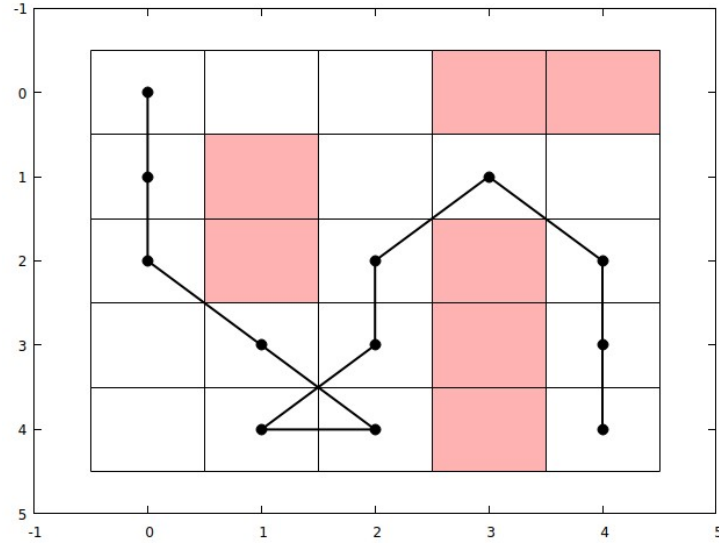


Figure 1: Path Found Using DFS

Nodes Explored	Length of Path	Cost of Path
13	12	13.0711

Table 1: Path Metrics

The returned values from the DFS algorithm contrast the optimal the path. The length of the path returned is 12 nodes, whereas the optimal length is 7 nodes. This can be seen in **Figure 2** below.

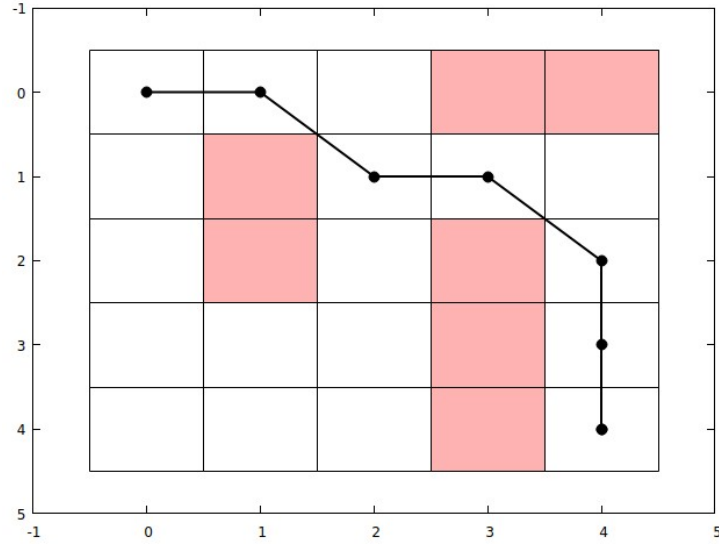


Figure 2: Optimal Path

2.2 Comparing DFS and Dijkstra

To compare the efficiency and effectivity between Depth First Search and Dijkstra's Algorithm, one could analyze the returned paths and how they vary in direction, number of nodes, cost, and execution time. This subsection will complete the comparison between the two search algorithms using the 10-by-10 occupancy grid

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The generated path connects node (0,0) and node (8,9).

Given the above occupancy grid, the returned paths for DFS and Dijkstra's along with their metrics are displayed in **Figure 3**, **Figure 4**, and **Table 2** respectively.

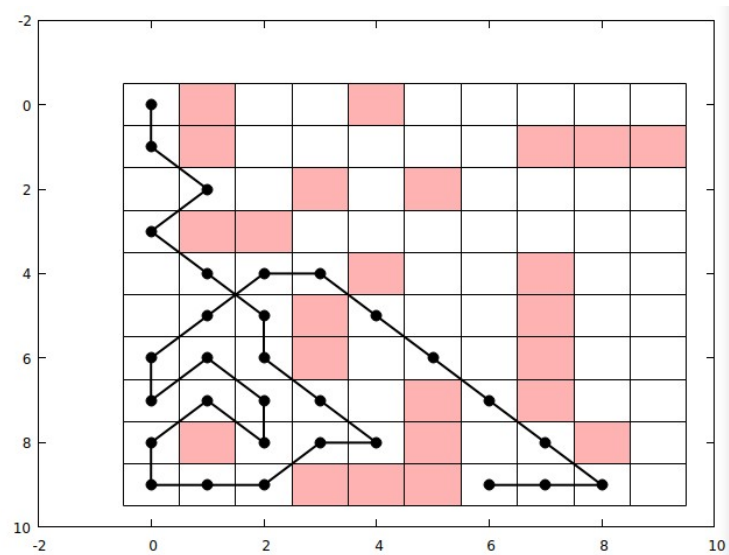


Figure 3: Path Found Using DFS: Medium Grid

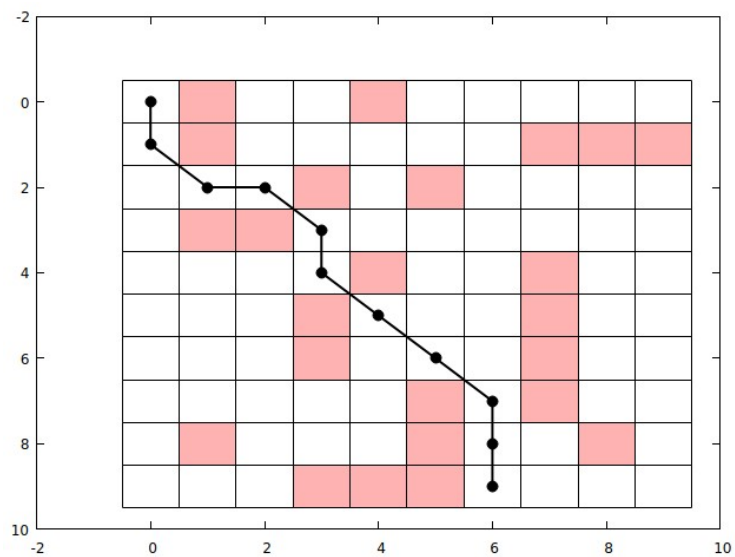


Figure 4: Path Found Using Dijkstra: Medium Grid

	Nodes Explored	Length of Path	Cost of Path	Execution Time (μs)
DFS	73.0	30.0	36.5	415.0
Dijkstra	67.0	11.0	12.1	531.0

Table 2: Path Metrics

Based on the data in both **Table 2**, **Figure 3**, and **Figure 4** it can be seen that at the scale of this grid, DFS is slightly faster than Dijkstra's, but finds a less much less optimal path. Dijkstra's found path is the most optimal, and costs less than a third of the DFS generated path.

2.3 Effects of A* Proceeding Cost Estimation

The heuristic function for A* defines the unique efficiency obtained in the path generation that does not appear in DFS and Dijkstra's. By estimating the proceeding cost for a given node (the cost between a node and the end node), the search algorithm can narrow the search for nodes that have promising prospects to lead to the shortest path. The behavior of a **greedy algorithm** (one that attempts to make intelligent decisions between nodes) is dictated by this estimation of proceeding cost. The following figures and tables attempt to document the effects of variation to this estimation through underestimation, overestimation, and "equal" estimation.

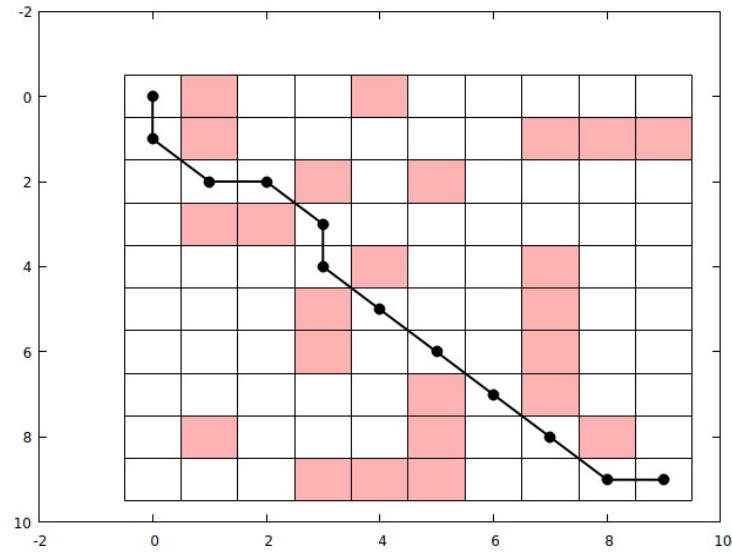


Figure 5: Underestimated Heuristic

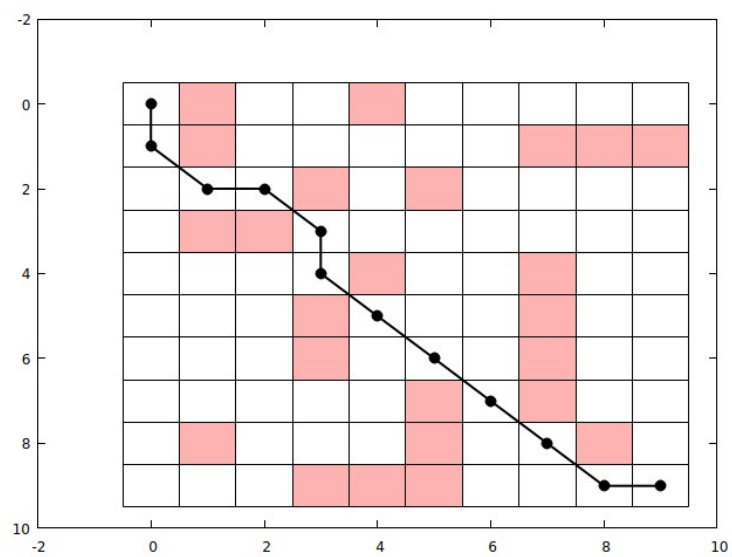


Figure 6: Equal Estimation Heuristic

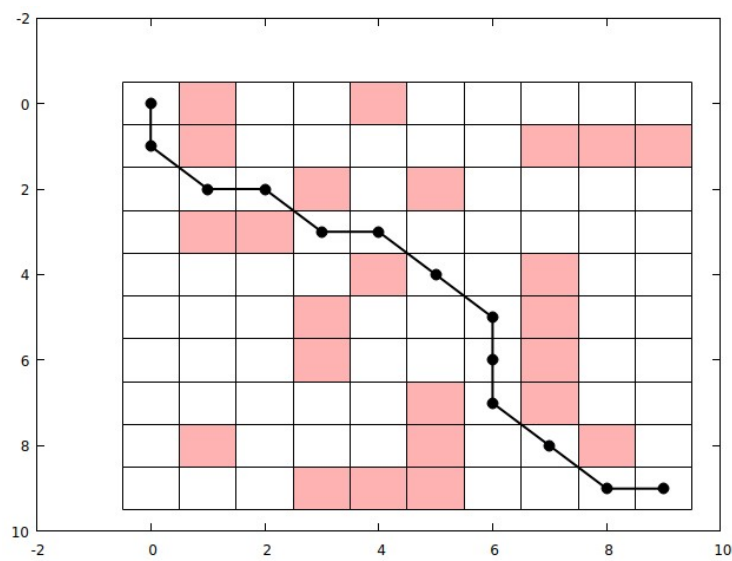


Figure 7: Overestimated Heuristic

	Nodes Explored	Length of Path	Cost of Path	Execution Time (μ s)
Underestimate	78.0	12.0	13.9	96.0
Equal	20.0	12.0	13.9	33.0
Overestimate	12.0	13.0	14.5	25

Table 3: Path Metrics

By implementing heuristic functions that underestimate, overestimate, and equally estimate, the data presented in **Figure 5**, **Figure 6**, **Figure 7**, **Table 3** could be generated. The A* algorithm was tasked with finding the shortest path between nodes (0,0) and (9,9). In the table, metrics for each path can be found.

Based on the data, it can be seen that by increasing the estimation of the proceeding cost of a node leads to a more strict exploration protocol. This makes sense, considering the priority queue currently implemented compares the heuristic value summed to the current cost of a node. By over estimating the heuristic value, the importance of the preceding cost is lessened, directing exploration directly to the end node. The weakening of preceding cost explains why the underestimated algorithm explored 78 nodes while the overestimated algorithm explored only 12 nodes.

The drawback to a more narrow exploration pattern is that the algorithm may not reevaluate the preceding cost to a node, saving a non-optimal path to a given node. That could explain why the overestimated heuristic resulted in a non-optimal path to the end node.

Based on the figures and table, a heuristic function that can accurately estimate the expected cost of a node is valuable in both efficiency and effectivity in determining the shortest path.

2.4 Determining Velocity, Acceleration, and Yaw Time Histories

The current implementation of A* described in **Section 3.3** only returns a time history of position in the path generated. However, the simulator developed in [3] takes inputs of position, velocity, acceleration, and yaw time histories.

In order to have a compatible pipeline between the path finder and simulator, the path finder must be modified to further generate these samples.

One way velocity and acceleration can be generated is to make velocity a function of the distance from the path changing direction. When the path changes direction (in x or in y), the velocity on that axis must become 0 (without polynomial smoothing discussed later). Since the velocity can be valued at 0 at every direction change, the velocity between each path change can contain 3 regions bounded by distance.

First, an attack region, where the velocity steadily or exponentially increases. This attack can be defined to safely accelerate the quad while still maximizing path time. The attack region can also be bounded by the sustain value.

Second, a sustain region, where the velocity is constant (acceleration is 0). The sustain value can either be set as the maximum velocity for the quad, or as a function of distance (where max regional velocity increases with distance between direction changing nodes).

Third, a decay region, where the velocity steadily or exponentially decreases towards the 0 velocity needed to change direction.

By defining these regions between path changing nodes, we allow the quad to have a generated position, velocity, and acceleration time history from the A* Algorithm.

For the yaw time history, the quad can be defined to always face the direction of motion. The pitch of the quad can be a function of velocity (described above). The roll in this case can be set to 0. These values would allow for the generation of a time history of quad attitude.

While the method described above would generate a time history for position, velocity, acceleration, and yaw, the complexity of implementation and lack of efficiency in the resulting path would make this implementation undesirable in practice. The A* algorithm needs to execute in a timely manner to update changes in path mid flight, so other methods of generated time histories (such as polynomial smoothing and sampling) are better options.

3 Implementation

The implementation of these search algorithms can generally be differentiated by the structure of the frontier (nodes to explore). The order of exploration changes the efficiency of the algorithm and the effectiveness of determining the shortest path.

Looking past the frontier structure, all three algorithms follow similar patterns. Each initially follows a loop that explores nodes (in the order of their respective frontier), where the neighbors of the node being explored are appended into the frontier. The explored node is released from the frontier, and the next node in line is prepared for exploration. This cycle continues until the node to be explored is the end-node.

Then, all three algorithms follow the same path generation method. In exploration, all nodes added to the frontier are given a reference to their "parent node", or the node currently being explored. This feature allows for a reverse traversal of a linked list, beginning at the end-node, generating a backward path. After reversing the order, the path from a starting node to an ending node is made.

For more information on the resulting metrics of these algorithms, and how they compare to each other, refer to **Section 2**.

3.1 Depth First Search

Depth First Search (DFS) is defined by its Stack data structure for the frontier. In other words, the node most recently found is the next node to be explored. This is not always the case however, as DFS does not allow for the re-exploration of nodes. In contrast to the next two algorithms, DFS does not account for cost. This leads to the behavior described above in the 2.1.

3.2 Dijkstra's Search Algorithm

Dijkstra's algorithm utilizes a priority queue that brings the least costly nodes to the front of the line. In this case, cost refers to the cost to travel from the starting node to the frontier node. This method of ordering the nodes in the frontier effectively determines the shortest path from a starting to an ending node. The main drawback is the execution time, which is much higher than DFS and A* due to the larger quantity of nodes that are explored to before reaching the end node.

3.3 A* Search Algorithm

Just as Dijkstra's, A* uses a priority queue to structure the frontier. The difference is the priority function. With A*, not only is the cost compared, but also a **heuristic** value. The heuristic can be determined a number of ways, and is used to direct the search towards nodes with the most promising solution. In this implementation, the heuristic is defined be the euclidean distance from the selected node to the end node. This can be found with the following equation:

$$H = \sqrt{(x_f - x_s)^2 + (y_f - y_s)^2}$$

Where H is the returned heuristic value for a node, x_f and y_f are the respective end node x and y positions and x_s and y_s are the respective selected node x and y positions.

This function allows for a very efficient and effective algorithm that returns the shortest path and minimizes the number of nodes explored.

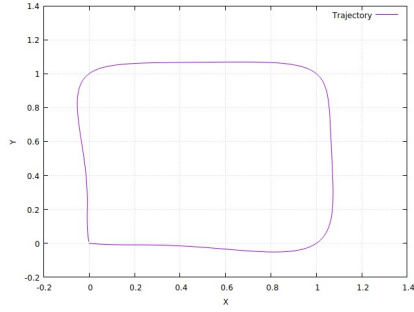
4 Experimenting with Polynomial Smoothing

In this section, different characteristics of the polynomial solver (described in depth in [6]) are varied to observe the effects of these characteristics. Each subsection highlights different variations, and contains plots of the path generated. Basic path way points are passed through to the polynomial solver for testing.

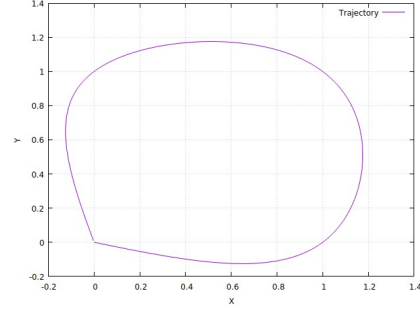
4.1 Effects of Derivative Variation

In this subsection, a quasi-square 2D trajectory is created by passing in 4 way points that create a square path.

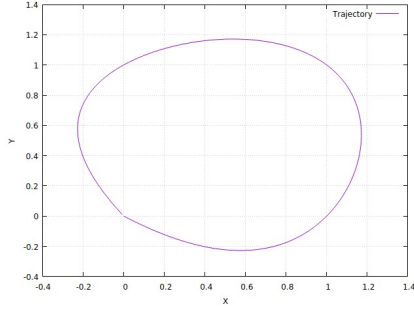
After defining the path, the configuration options for the polynomial solver, specifically the derivative order, can be altered. Let it be known the derivative order must be at least 3 less than the order of the polynomial. In this case, the polynomial order is a constant 8. So, the variation of the derivative order is constrained in a range $[1,5]$. Our test points can contain all 5 increments. **Figure 8** displays these 5 increments as (a), (b), (c), (d), (e) respectively, where the derivative order is expressed as D .



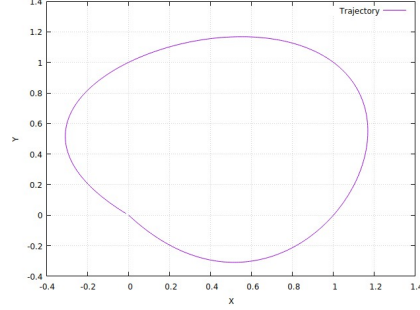
(a) $D = 1$



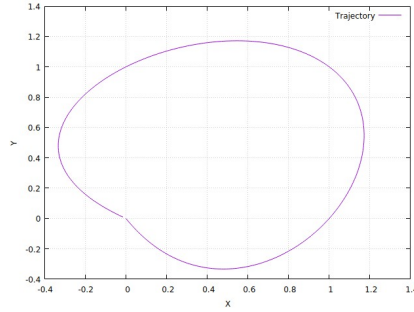
(b) $D = 2$



(c) $D = 3$



(d) $D = 4$



(e) $D = 5$

Figure 8: Effects of Order of Derivative D

Based on the generated paths after varying the order of the derivative, an increase in the derivative order seems to correlate to an increase in the smoothing of the path. In **(a)**, when $D = 1$, the path is very close to the theoretical square trajectory. But as D increases to 5, the path continues to take wider arcs in pursuit of each constrained node.

4.2 Effects of Arrival Time Variation

The quasi-square path example is continued in this subsection as used in **section 4.1**. Now, instead of changing the derivative, the experiment will vary the arrival time. The three different test will observe the effects of **(1)** a reasonable arrival time, **(2)** an unreasonably short arrival time, and **(3)** the shortest arrival time you can find with velocity and acceleration constrained to the follows bounds: $v(t) \leq v_{max} = 2 \text{ m/s}$ and $a(t) \leq a_{max} = 0.4 \text{ m/s}^2$. For (1) and (2), the between-waypoint time increments are uniform. For (3), the time increments may be non-uniform.

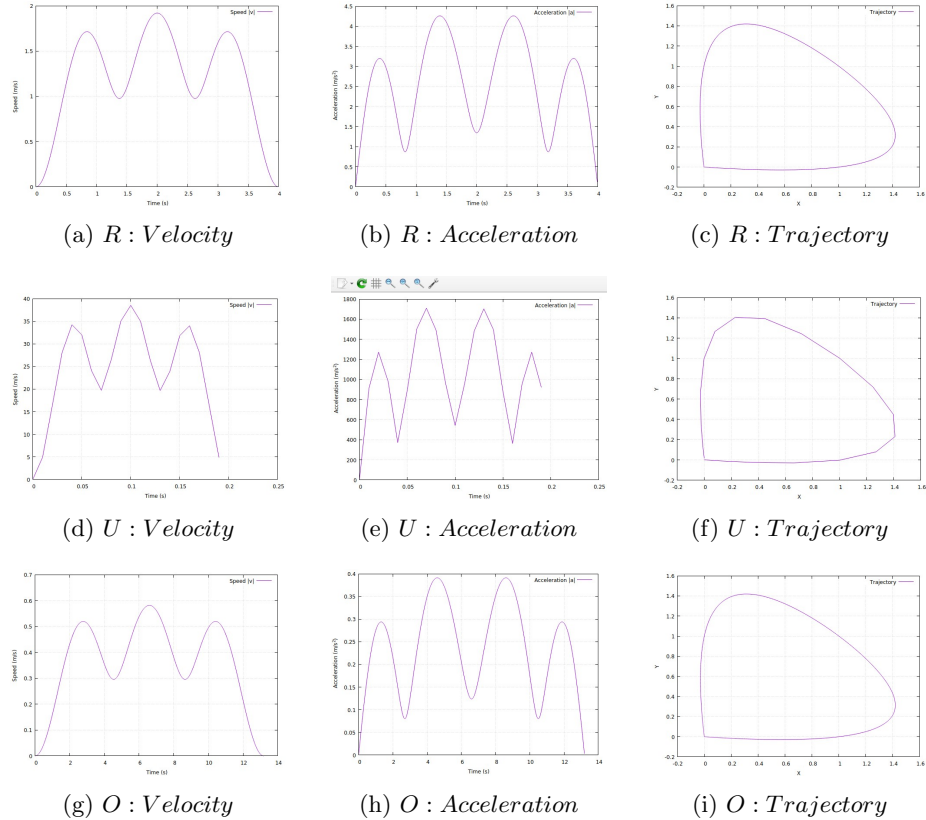


Figure 9: Effect of parameter k on system behavior

In **Figure 9**, the velocity, acceleration, and trajectory profiles are displayed for reasonable(**R**), unreasonable (**U**), and optimal (**O**) time increments. The time increments were as follows: $R : [0, 1, 2, 3, 4]$, $U : [0.00, 0.05, 0.10, 0.15, 0.20]$, $O : [0.0, 3.3, 6.6, 9.9, 13.2]$.

Based on the graphs, a noticeable difference can be seen between the reasonable and unreasonable arrival times between each node. In the reasonable scenario, the velocity, acceleration, and trajectory maintain a smooth transition throughout the entire period. In contrast, the unreasonable scenario graphs appear to have sharper changes in value. The acceleration and velocity experienced in the unreasonable scenario is also very large, which is to be expected when decreasing the arrival times to very low values.

The values for the optimal arrival times were found through brute force and binary search. In verification for the constraints, it was found that the quad would be limited by the max acceleration of 0.4 m/s^2 , where the max velocity is not approached. The resulting trajectory is still near identical to the ones found under reasonable and unreasonable arrival times, but shares the smoother quality of reasonable. This was expected, since the arrival times are much more achievable in the optimized scenario.

4.3 Effects of Varying Amount of Way Points

In this subsection, rather than defining a quasi-square path to follow, the experiment follows a circular path under a variable number of points. The implementation utilizes the relationship of trigonometric functions, attempting to generate a path around a center point with a constant radius of 1.

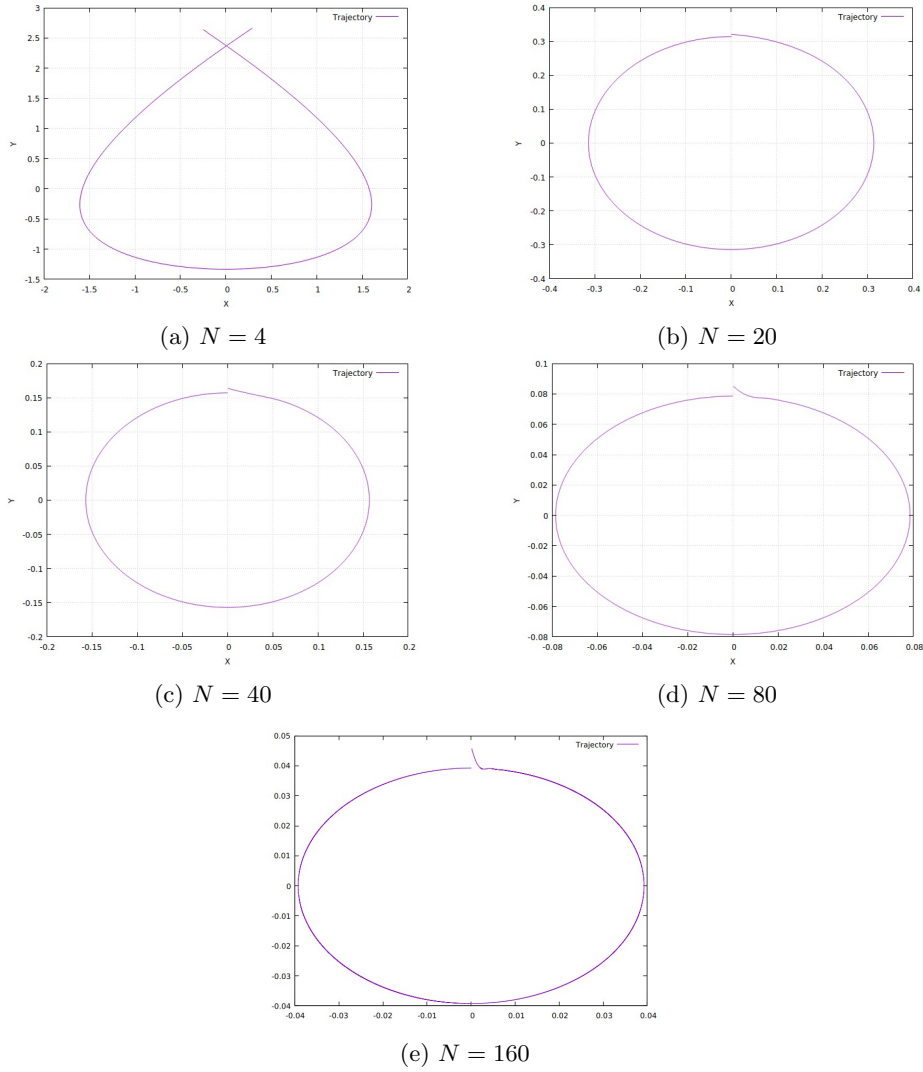


Figure 10: The Effects of N on the Trajectory

Figure 10 describes the trajectory generated after polynomial smoothing for circular paths containing N points. N is given values 4, 20, 40, 80, 160. The initial value of 4 was chosen for two reasons. First, the previous experiments utilized 4 points to generate a square, and I wanted to view the variation between the circular path and quasi-square path with the same number of points. Second, 4 was the minimum number of points for a quasi-circular path to generate properly. The next values were chosen in a binary manner, beginning with 20 and increasing to 160.

There are three difference seen in these graphs as the number of points

increases. It is first noted that the smoothness of the circular path is increasing with the number of points, while also becoming a more defined oval. Second, the lip appearing at the top of the trajectory (when y is at a maximum) appears to increase with the number of points. Third, the magnitude of the radius of the path decreases with the number of points. This decrease appears linear, and it is not clear where in the implementation this phenomenon appears.

5 Results

5.1 Polynomial Smoothing Integrated to A*

By pipelining one of the path finding algorithms described in **Section 3** to the polynomial solver, one could generate a smoothed path that is generated intelligently (contrasting the experiments in **Section 4**). In this section, the pipeline utilizes A* to generate the path, and the smoothed path is exported into the simulator described in detail in [3] and [4].

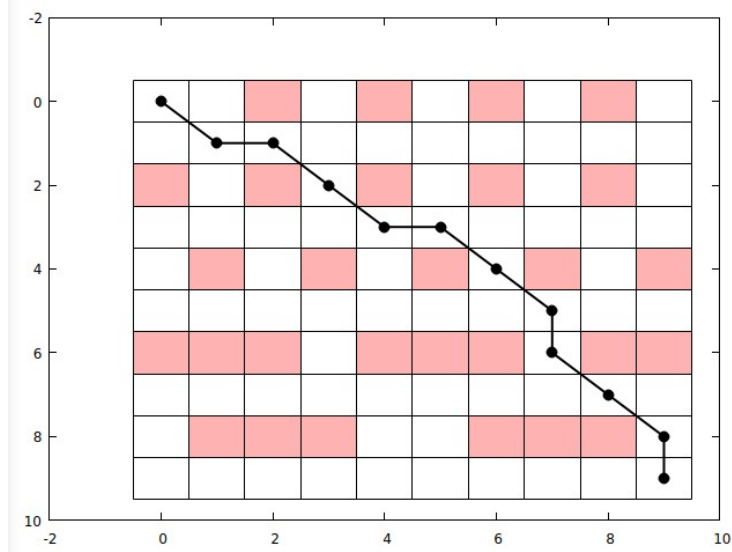


Figure 11: A* Path Generated

In **Figure 11** above, A* was tasked with finding the shortest path from node (0,0) to node (9,9) on the full stack grid. After running polynomial smoothing, the following figure describes the found trajectory:

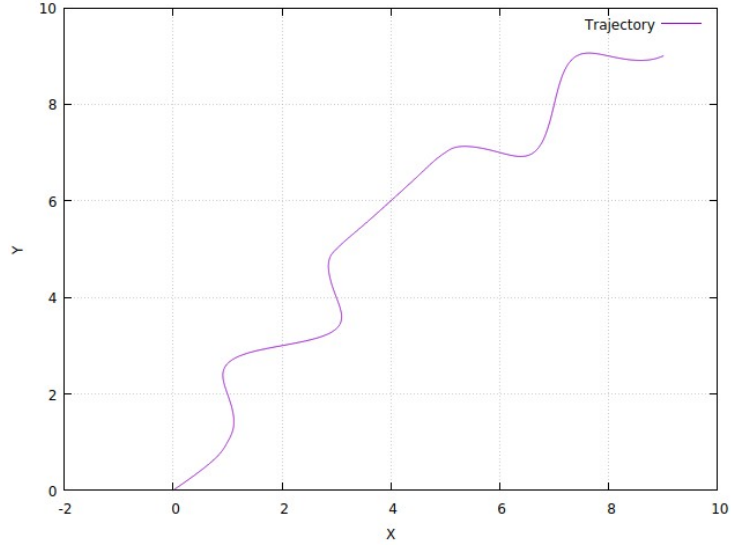


Figure 12: Path After Polynomial Smoothing

Figure 12 is the found trajectory to be exported out of the full stack pipeline. It is worth noting that the orientation varies slightly from **Figure 11** due to the location of the origin in both figures. The top left node in the grid is considered $(0,0)$, which is represented on the trajectory plot at the origin. The exported position, velocity, and acceleration files consisted of a 3×2400 entry matrix, resulting from the 12 second period (1 second arrival increments for each of the 12 nodes) and the 200 Hz sample rate.

By removing the first row of the matrix (time row vector), transforming the matrices, appending a zero vector for the z axis, and generating a yaw matrix through velocity normalization, the necessary inputs to the high fidelity control simulator were available.

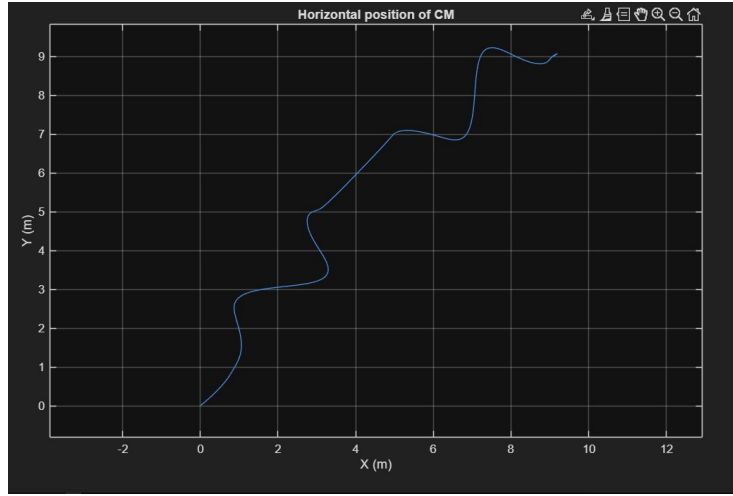


Figure 13: Simulated Control Path with Estimation

Figure 13 displays that outputted trajectory of the simulator quadrotor with estimation. This trajectory was fed from the path generation pipeline, and would allow the quad to traverse the full stack grid from node (0,0) to (9,9). The trajectory is nearly identical to the desired trajectory, highlighting the accuracy of the simulator built in [4]. It should be noted that the z axis remained a constant 0, due to the path finder working in 2D space.

6 Conclusion

This report follows the analysis and implementation of three path finding algorithms: Depth First Search (DFS), Dijkstra’s Algorithm, and A-star (A*). Their individual benefits and characteristics were highlighted in **sections 2** and **3**.

Past the path finding algorithms, polynomial smooth can be used to further generate a more suitable path for a quad to follow. This increases efficiency of execution time with a more natural path. Furthermore, it allows for higher order constraint to the time history generation (accounts for velocity and acceleration).

By pipelining these features, an effective path in 2D space can be generated and made digestible by the high fidelity quadrotor simulator developed in [2], [3], and [4].

References

- 1 Humphreys, Todd. “Main.” Jan. 2026.
- 2 Vanbrocklin, Jett. ”Simulating Quadrotor Dynamics” Feb. 2026.

- 3 Vanbrocklin, Jett. "Simulator Refinements and Quadrotor Control" Feb. 2026.
- 4 Vanbrocklin, Jett. "Measurement Simulation and State Estimation" Mar. 2026.
- 5 Humpreys, Todd. "lab4" Mar. 2026.
- 6 Humpreys, Todd. "Piecewise Polynomial Path Planner (P4)" Mar. 2020.